TP 1 Projet Fédéré

Préparation de l'environnement de travail

Objectif : Configurer l'intégration continue d'une chaîne d'outils et mettre en œuvre un workflow de développement agile.

Description :

Au terme de ce TP, vous serez capables de :

- Configurer un environnement collaboratif GitHub + Jira
- Maîtriser les commandes Git essentielles
- Synchroniser les commits GitHub → Tickets Jira automatiquement

Etape 1 : Configuration initiale

Création d'un compte Jira

1. https://www.atlassian.com/fr/software/jira → "Essayer gratuitement"
2. Email : votre email → ***NB : utiliser la même adresse pour la création du compte Github aussi***
3. Créer projet SCRUM et choisir un nom par exemple "SI2-PF-Team1"
4. Créer un token API (pour l'integration avec GitHib après)
   a. Cliquer sur ton avatar en haut à droite → Paramètres du compte → Sécurité.
   b. Descendre jusqu'à Jetons API (API tokens).
   c. Cliquer sur Créer un jeton API.
   d. Donner-lui un nom, par exemple GitHub_Integration.
   e. Cliquer sur Créer.
   f. Copier le token dans un fichier .txt par exemple (on ne pourra pas le voir après).

NB : ce token sera utilisé plus tard comme **mot de passe** lors de l'intégration avec GitHub

Création d'un compte GitHub + Dépôt

1. Accéder au site github.com pour créer un compte
2. Remplir les champs obligatoires
3. Valider la création de votre compte
4. Créer un nouveau dépôt (new repository) :
   a. Nom : tp1-jira-github
   b. Visibilité : privée
   c. **Ne pas** cocher "Initialize with a README".
   d. Noter l'URL HTTPS du dépôt.

Configurer Git Windows

1. https://git-scm.com/download/win → Download
2. Installateur → Accepter les options par défaut
3. Vérifier installation : Windows Start → "Git Bash" → Ouvrir → taper : $ git --version

4. Configuration identité

```
git config --global user.name "…"
git config --global user.email "…"
git config --global core.autocrlf true
git config --list
```

5. Clés SSH : La clé SSH permet à votre ordinateur de se connecter de façon sécurisée à un serveur (comme GitHub) **sans avoir à saisir votre mot de passe à chaque fois**.

```
ssh-keygen -t ed25519 -C "votre email"
```

```
cat ~/.ssh/id_ed25519.pub  # Copier TOUT le contenu → ssh-ed25519 AAAAC3...
```

6. Ajouter SSH GitHub

GitHub > Account Settings > SSH and GPG keys > "New SSH key"
Title : "PF-TP" → Coller clé → Add SSH key

Test :

```
ssh -T git@github.com
```

Cloner le projet

1. Créer un dossier C/PF-TP
2. Accéder à ce dossier puis ouvrir un Git Bash
3. Taper :

```
git clone git@github.com:<votre_nom>/tp1-jira-integration.git
cd tp1-jira-integration
git status
```

Explication :

`git clone <url>`

- télécharge un dépôt GitHub
- crée le dossier tp1-jira-integration **localement**
- fait automatiquement **git init**
- ajoute le **remote origin** (`origin = git@github.com:houdapf2526/tp1-jira-integration.git`)
- récupère les branches et commits

`git status :`

- montre l'état du dépôt
- indique la branche courante
- indique s'il y a des fichiers à ajouter ou committer

Lier GitHub à Jira

1. Dans Jira : Apps > Explore more apps > Cherchez "GitHub for Atlassian" > Get app.
2. Suivre l'assistant pour connecter votre espace jira/dépôt GitHub.
3. Ouvrir le projet Jira Cloud.
4. A gauche de votre avatar :
   Icône Paramètres → Paramètres d'administrateur Jira → Apps du Marketplace → GitHub fo Atlassian → Configure

5. Sur la page « Connect GitHub to Atlassian », cliquer sur **Continue** → GitHub Cloud → Select a GitHub organization (Votre compte GitHub) → Connect.
6. Jira va vous rediriger vers GitHub pour **autoriser l'accès (**se connecter au compte GitHub si nécessaire).
7. Autoriser Jira à accéder aux **repository à lier**.
8. Sélectionner le dépôt tp1-jira-integration dans la liste.
9. Clique sur **Connect / Link**.

NB : Jira utilise votre **token API** pour certaines opérations, mais l'application officielle gère l'authentification via OAuth. Donc vous n'avez normalement pas besoin de retaper le token ici, juste de **donner l'autorisation GitHub → Jira**.

Etape 2 : Tester le lien GitHub ↔ Jira

Créer une tâche dans Jira

1. Dans le backlog Jira, créez une **"Story"**.
2. **Titre** : "**En tant que** visiteur, **je peux** accéder à la page d'accueil sans authentification, **afin de** parcourir le catalogue".
3. **Créer une tâche** : "Créer la page d'accueil"
4. **Description** : "Créer un fichier index.html".
5. **Copiez la clé de la tâche** (ex: `SCRUM-2`).

Créer les fichiers du projet

1. Créez un fichier `index.html` avec un contenu HTML simple.
2. Créez un dossier `docs/` et un fichier `architecture.puml` à l'intérieur. Collez-y ce code PlantUML :

```
@startuml
title Diagramme d'Architecture Simple
rectangle "Navigateur Web" as client
rectangle "Serveur GitHub Pages" as server
client -> server : Requête http
server --> client : Renvoie index.html
@enduml
```

3. Créez un fichier README.md et insérez-y une référence au diagramme :

```
![Diagramme](docs/architecture.puml)
```

Commiter et pousser les changements

<u>Initialisation (optionnelle mais recommandée):</u>

Dans le Git Bash, taper :

```
git add .
git commit -m "Initial commit - page d'accueil"
git push origin main
```

Explication

`git add :`

- dit à Git **quels fichiers on veut enregistrer**
- *. = tous les fichiers du dossier*

`git commit :`

- crée un **point de sauvegarde**
- `-m` = message du commit (obligatoire)

`git push :`

- git push : envoie les commits vers le dépôt distant
- origin : le dépôt GitHub
- main : la branche que tu envoie
- -u : dit à Git « *main local est lié à main sur GitHub* »
  (les prochains git push seront plus simples)

Commit lié au ticket Jira

Ajouter une ligne dans README.md pour montrer un changement :

```
echo "- Test liaison Jira" >> README.md
```

Ajouter les changements à Git

```
git add README.md
```

Créer le commit avec la clé Jira (Supposons que la clé de ton ticket soit SCRUM-1) :

```
git commit -m "SCRUM -2: Test liaison GitHub ↔ Jira"
```

Push du commit lié à Jira

```
git push
```

(Remarque : Comme on a déjà fait -u origin main au premier push, **plus besoin de répéter origin main)**

➔Le commit est poussé. Le message de commit contient la clé Jira SCRUM-1, ce qui crée un lien automatique dans Jira.

Etape 3 : Vérification, Pull Request et Transition de statut

Vérifier l'intégration dans Jira

1. Retournez sur la tâche SCRUM-1 dans Jira.
2. Cherchez la section "Développement". Vous devez voir un lien vers la branche et le commit que vous venez de pousser, preuve que l'intégration fonctionne.

Créer et fusionner une Pull Request (PR)

1. Créer une nouvelle branche locale (Travailler sur une fonctionnalité **sans toucher à main**.)

```
git checkout -b SCRUM-2-creer-page-groupe
```

Explication

- git checkout : changer de branche

- -b : crée une nouvelle branche **et** se déplace dessus
- SCRUM-2-creer-page-groupe : nom de la branche (idéalement lié à Jira)

2. Vérifier : git branch
3. Modifier des fichiers et faire un commit sur la branche
   - Ajouter un fichier *groupe.html*
   - Ajouter les changements à Git : git add groupe.html
   - Faire le commit lié à Jira : git commit -m "SCRUM-2: Création de la page groupe"
4. Pousser la branche sur GitHub
   - git push -u origin SCRUM-2-creer-page-groupe
5. Vérification
   - Va sur GitHub → ton repo
   - Tu dois voir **la branche** SCRUM-2-creer-page-groupe dans la liste des branches
   - Le commit avec le message SCRUM-2: Création de la page groupe doit être visible
6. Sur GitHub, créer une nouvelle PR pour fusionner votre branche dans main.
   - Ouvrir ton repo tp1-jira-integration
   - Cliquer sur l'onglet **Pull requests**
   - Cliquer sur **New pull request**
   - Sélectionner les branches
     - Base : main (branche dans laquelle tu veux fusionner)
     - Compare : SCRUM-2-creer-page-groupe (ta branche de fonctionnalité)
   - GitHub te montrera les commits qui seront fusionnés
   - Ajouter un titre et une description
     - Title : SCRUM-2: Création de la page groupe
     - Description : détaille ce qui est fait pour le ticket Jira
   - Jira affichera ensuite la PR dans la section *Développement*
   - Cliquer sur **Create pull request**
7. Fusionner ("Merge") la Pull Request : Clique sur **Merge pull request** (bouton vert).
8. Confirmer en cliquant sur **Confirm merge**
9. Supprimer la branche (optionnel mais recommandé) :

   ```
   git push origin --delete SCRUM-2-creer-page-groupe
   ```

10. Mettre à jour ta branche **locale** *main*

    ```
    git checkout main
    git pull
    ```

    - git checkout main : revenir sur main
    - git pull : récupérer les dernières modifications depuis GitHub

    Tous les commits de la PR sont maintenant dans main localement

Gérer la transition de statut

Règle d'Automatisation

1. Dans votre projet Jira, allez dans Paramètres de l'espace > Automatisation.
2. Créer une nouvelle règle avec le déclencheur "Pull request mergée" dans la catégorie DevOps.
3. Ajouter l'action "Transitionner le ticket" vers le statut "Terminé".
4. Activer la règle. La fusion de votre PR devrait maintenant déclencher la transition automatique.